

LAB_02_Thomas_Toledo_Travassos

Thomas Toledo Travassos

###Lab_02

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.3      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr      1.2.2
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(readr)
```

1. Quais são as estatísticas suficientes para a determinação do percentual de vôos atrasados na chegada (`ARRIVAL_DELAY > 10`)?

Para determinar o percentual de voos atrasados, são necessárias duas estatísticas para cada combinação de dia, mês e companhia aérea: o número total de voos e o número de voos cujo `ARRIVAL_DELAY` é superior a 10 minutos. O percentual será obtido pela razão entre o número de voos atrasados e o número total de voos

2. Crie uma função chamada `getStats` que, para um conjunto de qualquer tamanho de dados provenientes de `flights.csv.zip`, execute as seguintes tarefas (usando apenas verbos do `dplyr`):
 - a. Filtre o conjunto de dados de forma que contenha apenas observações das seguintes Cias. Aéreas: AA, DL, UA e US;

- b. Remova observações que tenham valores faltantes em campos de interesse;
 - c. Agrupe o conjunto de dados resultante de acordo com: dia, mês e cia. aérea;
 - d. Para cada grupo em b., determine as estatísticas suficientes apontadas no item 1. e os retorne como um objeto da classe tibble;
 - e. A função deve receber apenas dois argumentos: I. input: o conjunto de dados (referente ao lote em questão);
- II. pos: argumento de posicionamento de ponteiro dentro da base de dados. Apesar de existir na função, este argumento não será empregado internamente. É importante observar que, sem a definição deste argumento, será impossível fazer a leitura por partes.

```
getStats <- function(input, pos) {

  input %>%
    filter(AIRLINE %in% c("AA", "DL", "UA", "US")) %>%
    filter(!is.na(ARRIVAL_DELAY)) %>%
    mutate(atrasado = ARRIVAL_DELAY > 10) %>%
    group_by(DAY, MONTH, AIRLINE) %>%
    summarise(
      total = n(),
      atrasados = sum(atrasado),
      .groups = "drop"
    )

}
```

- 3. Utilize alguma função readr::read_***_chunked para importar o arquivo flights.csv.zip.
 - a. Configure o tamanho do lote (chunk) para 100 mil registros;
 - b. Configure a função de callback para instanciar DataFrames aplicando a função getStats criada acima;
 - c. Configure o argumento col_types de forma que ele leia, diretamente do arquivo, apenas as colunas de interesse (veja nota de aula para identificar como realizar esta tarefa);

```
stats_lotes <- read_csv_chunked(
  "datasets/flights.csv",
  callback = DataFrameCallback$new(getStats),
  chunk_size = 100000,
  col_types = cols_only(
    MONTH = col_integer(),
```

```

    DAY = col_integer(),
    AIRLINE = col_character(),
    ARRIVAL_DELAY = col_double()
  )
)

```

4. Crie uma função chamada `computeStats` que:

- a. Combine as estatísticas suficientes para compor a métrica final de interesse (percentual de atraso por dia/mês/cia aérea);
 - b. Retorne as informações em um tibble contendo apenas as seguintes colunas: I. Cia: sigla da companhia aérea;
- II. Data: data, no formato AAAA-MM-DD (dica: utilize o comando `as.Date`);
- III. Perc: percentual de atraso para aquela cia. aérea e data, apresentado como um número real no intervalo [0,1]

```

computeStats <- function(stats) {

  stats %>%
    group_by(DAY, MONTH, AIRLINE) %>%
    summarise(
      total = sum(total),
      atrasados = sum(atrasados),
      .groups = "drop"
    ) %>%
    mutate(
      Cia = AIRLINE,
      Data = as.Date(
        paste(2015, MONTH, DAY, sep = "-")
      ),
      Perc = atrasados / total
    ) %>%
    select(Cia, Data, Perc)

}

```

```
stats <- computeStats(stats_lotes)
```

```
stats
```

```
# A tibble: 1,276 x 3
  Cia    Data      Perc
  <chr> <date>    <dbl>
1 AA    2015-01-01 0.373
2 DL    2015-01-01 0.0847
3 UA    2015-01-01 0.289
4 US    2015-01-01 0.196
5 AA    2015-02-01 0.201
6 DL    2015-02-01 0.203
7 UA    2015-02-01 0.256
8 US    2015-02-01 0.327
9 AA    2015-03-01 0.541
10 DL   2015-03-01 0.372
# i 1,266 more rows
```

5. Produza um mapa de calor em formato de calendário para cada Cia. Aérea.
 - a. Instale e carregue os pacotes ggcal e ggplot2.
 - b. Defina uma paleta de cores em modo gradiente. Utilize o comando `scale_fill_gradient`. A cor inicial da paleta deve ser `#4575b4` e a cor final, `#d73027`. A paleta deve ser armazenada no objeto `pal`.
 - c. Crie uma função chamada `baseCalendario` que recebe 2 argumentos a seguir: `stats` (tibble com resultados calculados na questão 4) e `cia` (sigla da Cia. Aérea de interesse). A função deverá: I. Criar um subconjunto de `stats` de forma a conter informações de atraso e data apenas da Cia. Aérea dada por `cia`.
- II. Para o subconjunto acima, montar a base do calendário, utilizando `ggcal(x, y)`. Nesta notação, `x` representa as datas de interesse e `y`, os percentuais de atraso para as datas descritas em `x`.
- III. Retornar para o usuário a base do calendário criada acima.
 - d. Executar a função `baseCalendario` para cada uma das Cias. Aéreas e armazenar os resultados, respectivamente, nas variáveis: `cAA`, `cDL`, `cUA` e `cUS`.
 - e. Para cada uma das Cias. Aéreas, apresente o mapa de calor respectivo utilizando a combinação de camadas do `ggplot2`. Lembre-se de adicionar um título utilizando o comando `ggtitle`. Por exemplo, `cXX + pal + ggtitle("Título")`.

Exercício 5

```

library(ggcal)
library(ggplot2)

pal <- scale_fill_gradient(
  low = "#4575b4",
  high = "#d73027"
)

baseCalendario <- function(stats, cia) {

  dados <- stats %>%
    filter(Cia == cia)

  ggcal(dados$Data, dados$Perc)
}

cAA <- baseCalendario(stats, "AA")
cDL <- baseCalendario(stats, "DL")
cUA <- baseCalendario(stats, "UA")
cUS <- baseCalendario(stats, "US")

```

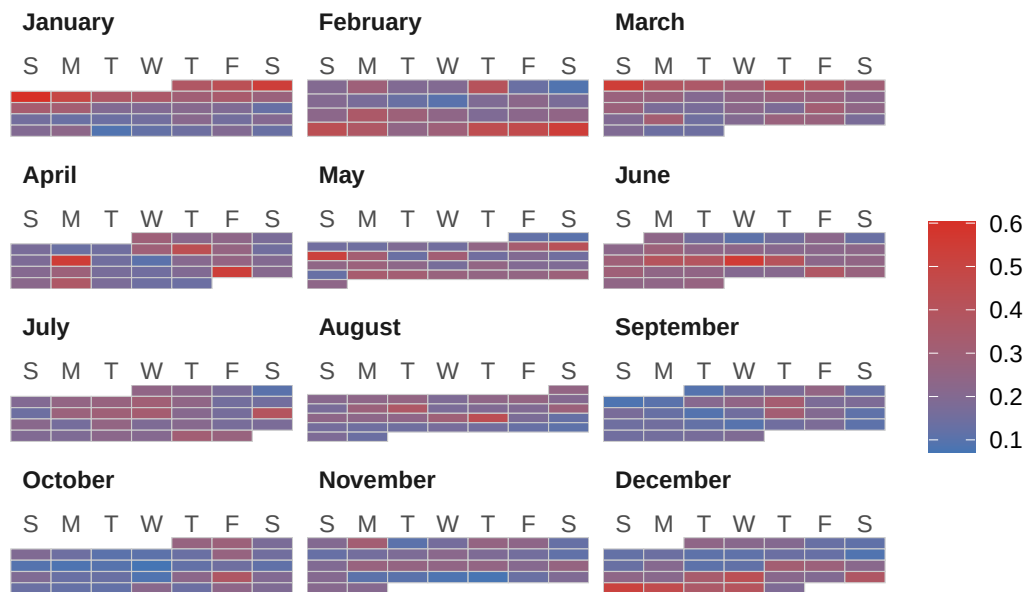
American Airlines

```

cAA + pal + ggtitle("American Airlines (AA)")

```

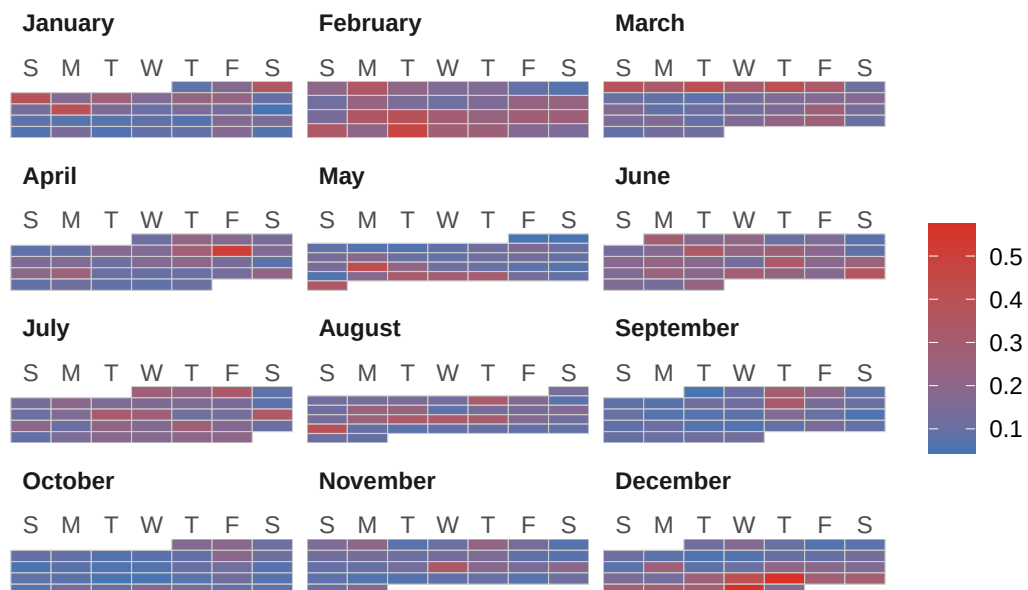
American Airlines (AA)



Delta Air Lines

```
cDL + pal + ggtitle("Delta Air Lines (DL)")
```

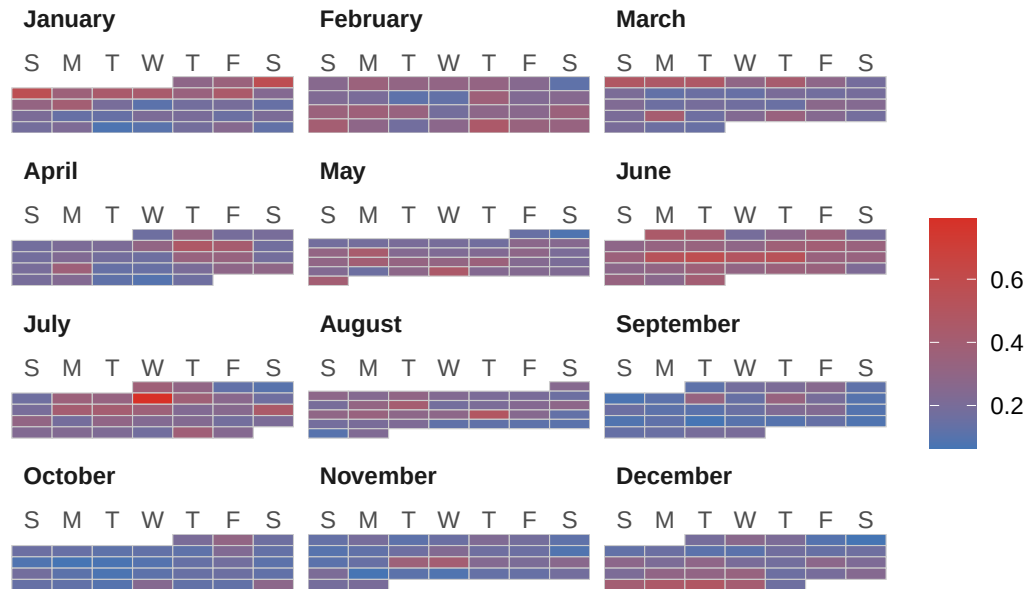
Delta Air Lines (DL)



United Airlines

```
cUA + pal + ggtitle("United Airlines (UA)")
```

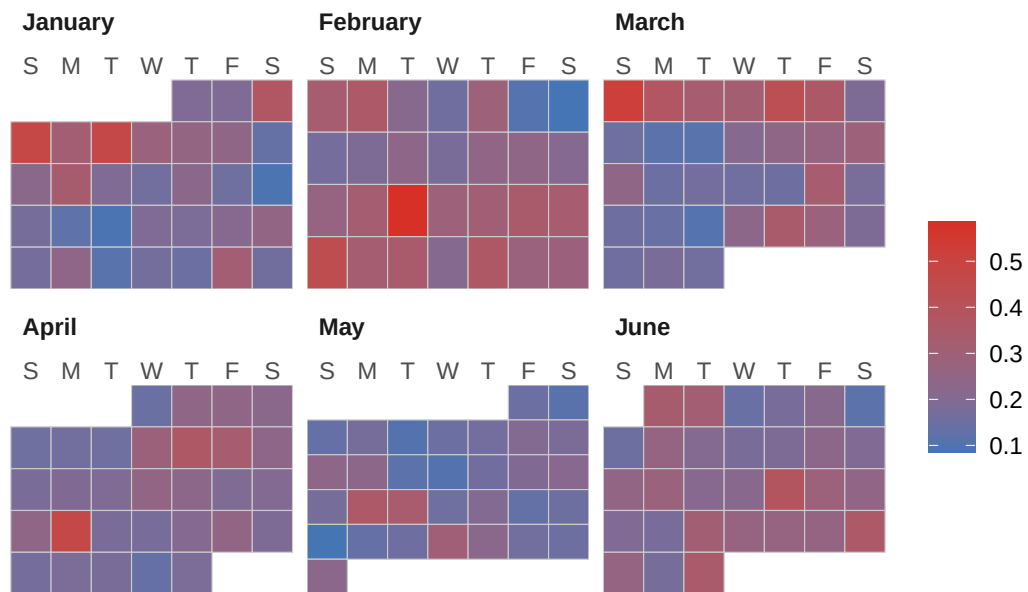
United Airlines (UA)



US Airways

```
cUS + pal + ggtitle("US Airways (US)")
```

US Airways (US)



```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-08-23 13:40:30 -03"
```